

Tema 4: Procesos, Fork, Exec y Wait

Este tema es el mas preguntado en los exámenes de Sistemas Operativos, representando entre un quince y un veinte por ciento de las preguntas. Tienes que dominarlo completamente.

Que es fork

Fork es la llamada al sistema que crea un nuevo proceso. Es la unica forma de crear procesos en Unix y Linux. Cuando un proceso llama a fork, se crea una copia casi exacta del proceso. El proceso original se llama padre, y la copia se llama hijo. Ambos continuan ejecutando desde la instruccion siguiente al fork.

Lo mas importante que debes recordar es el valor de retorno de fork. En el proceso padre, fork devuelve el PID del hijo, que es un numero mayor que cero. En el proceso hijo, fork devuelve cero. Si hay un error, fork devuelve menos uno. Esta es la forma de distinguir el padre del hijo: el padre recibe el PID del hijo, el hijo recibe cero.

Un patron tipico es: hacer fork, y luego un if. Si el retorno es cero, estamos en el hijo. Si es mayor que cero, estamos en el padre. Si es menos uno, hubo error.

Contar procesos con fork

Cada fork duplica todos los procesos existentes. Si empiezas con un proceso y haces un fork, tienes dos. Si haces otro fork, ambos procesos se duplican y tienes cuatro. Con un tercer fork, ocho.

La formula general es: si haces N forks consecutivos, terminas con dos elevado a N procesos. Ejemplo de examen: un for de cero a tres que solo contiene fork. Al terminar, hay dos elevado a tres igual a ocho procesos, y cada uno imprime la misma linea. El resultado son ocho lineas.

Otro ejemplo: printf de A, luego fork, luego fork, luego printf de B. El primer printf se ejecuta antes de cualquier fork, asi que A se imprime una vez. Luego el primer fork crea dos procesos, el segundo fork crea cuatro. Los cuatro imprimen B. Resultado: una A y cuatro B.

Cuidado con printf y el buffer

Hay una trampa sutil. Si usas printf sin salto de linea, el texto queda en el buffer. Cuando haces fork, el buffer se copia al hijo. Asi que tanto padre como hijo tendran el texto en su buffer, y cuando el buffer se vuelque, ambos lo escribirán. Esto puede hacer que A aparezca mas veces de las esperadas. Para evitarlo, usa fprintf a stderr, que no tiene buffer, o pon un fflush de stdout antes del fork.

La llamada exec

Exec reemplaza el programa de un proceso por otro programa. No crea un proceso nuevo: el mismo proceso cambia su codigo, datos y pila por los del nuevo programa. Si exec tiene exito,

no retorna nunca, porque el programa original ha sido reemplazado. Si `exec` falla, devuelve menos uno.

Esto es pregunta clasica de examen: si despues de `exec` hay un `printf`, ese `printf` nunca se ejecutara si `exec` tiene exito. Solo se ejecuta si `exec` falla.

Hay varias variantes de `exec`. `execl` recibe los argumentos como cadenas individuales terminadas en `NULL`. `execv` recibe un array de cadenas. `execlp` y `execvp` buscan el ejecutable en el `PATH`.

Un ejemplo de `exec` correcto: `execl de barra bin barra ls, comilla ls, comilla barra tmp, NULL`. Esto ejecuta `ls barra tmp`. El ultimo argumento debe ser `NULL` como terminador.

Un error comun que preguntan es poner `exec` en un bucle sin `fork`. El `exec` reemplaza el proceso, asi que el bucle no se ejecutara nunca mas. El `while` y el `if` posterior al `exec` sobran, porque si `exec` tiene exito, nada de eso se ejecuta.

El patron correcto es: `fork` para crear un hijo, y el hijo hace `exec` para ejecutar otro programa. El padre puede hacer `wait` para esperar a que el hijo termine.

La llamada `wait`

`Wait` hace que el padre espere a que un hijo termine. Recibe un puntero a un entero donde se almacena el estado de salida del hijo. Con la macro `WIFEXITED` puedes comprobar si el hijo termino normalmente, y con `WEXITSTATUS` obtienes el codigo de salida.

Si el padre hace `fork` y luego `wait`, y el hijo ejecuta algo y termina, el padre continuara despues de que el hijo termine. Esto garantiza un orden: primero el hijo, luego el padre.

Ejemplo de examen: `fork`, el hijo hace `printf` de hola y `exit`, el padre hace `wait` y luego `printf` de adios. El orden esta garantizado: siempre sale hola antes que adios, porque `wait` bloquea al padre hasta que el hijo termina.

Procesos zombies

Cuando un proceso hijo termina pero su padre no ha llamado a `wait`, el hijo se convierte en un proceso zombie. El zombie ocupa una entrada en la tabla de procesos del kernel pero ya no consume recursos de CPU ni memoria. Se elimina cuando el padre llama a `wait`.

Si no quieres esperar por los hijos, puedes hacer `signal` de `SIGCHLD` con `SIG_IGN`. Esto le dice al kernel que recolecte automaticamente los hijos terminados sin necesidad de `wait`.

`getpid` y `getppid`

`getpid` devuelve el PID del proceso actual. `getppid` devuelve el PID del padre. En un ejercicio de examen con dos `forks`, los cuatro procesos resultantes tendran distintos `getppid`. Los que fueron creados por el primer `fork` comparten el mismo padre original. Los que fueron creados por el segundo `fork` tienen como padre al proceso que los creo en el primer `fork`.

Ejemplo: fork, fork, printf de getppid. Los cuatro procesos escriben cuatro numeros. Siempre habra dos numeros iguales y otros dos distintos, porque los procesos hermanos comparten padre.

Variables de entorno

Las variables de entorno son pares nombre-valor que el sistema operativo pasa a los procesos. Se acceden desde C con getenv, pasando el nombre de la variable sin el signo dolar. Si quieres obtener el home del usuario, usas getenv de comilla HOME comilla, no getenv de comilla dolar HOME comilla. El dolar solo se usa en el shell.

La variable PATH contiene las rutas de los directorios donde se buscan los ejecutables, separados por dos puntos. La shell la usa para encontrar comandos cuando escribes su nombre sin ruta completa.

Las variables de shell no se heredan automaticamente a los procesos hijos. Solo se heredan si las exportas con export. Ejemplo de examen: defines mipid igual a veinte en el shell, luego abres un nuevo bash. En el nuevo bash, echo de dolar mipid no imprime nada, porque la variable no fue exportada.

El patron completo: fork mas exec mas wait

El patron completo para ejecutar un comando externo desde tu programa es: crear un pipe si necesitas comunicacion, hacer fork para crear un hijo, en el hijo cerrar descriptores innecesarios y hacer las redirecciones necesarias con dup2, luego exec para ejecutar el comando. En el padre, cerrar los descriptores que no usa y hacer wait para esperar al hijo.

Esto es exactamente lo que hace el shell cada vez que ejecutas un comando. Crea un proceso hijo con fork, redirige entrada y salida si es necesario, y ejecuta el programa con exec. Mientras tanto, el padre espera con wait, a menos que sea un proceso en background.